Automatic Short-Answer Grading using Corpus-based Semantic Similarity Measurements

Bhuvnesh Chaturvedi[1], Rohini Basak[1]

[1] Jadavpur University, Kolkata, India

bhuvneshchaturvedi2512@gmail.com, visitrohinihere@gmail.com

Abstract. In this paper, we have explored some unsupervised techniques for the task of Automatic Short Answer Grading. Three models are designed for the task in hand, with each model evaluating and grading students' responses individually. The proposed method has shown quite promising correlation between scores generated by the method and that awarded by human scorers on a standard computer science dataset, with the overall correlation reaching the peak value at 0.805 which outperforms the state-of-the-art results reported on this dataset so far.

Keywords: Automatic short answer grading, semantic similarity, dependency parser, POS tagger.

1 Introduction

The task of Automatic Short Answer Grading (ASAG) involves scoring a student answer (SA) based on a given reference (model) answer (RA or MA). Optionally, scoring schemes may also be provided to indicate relative importance of different parts of the model answer. This is a complex natural language understanding task due to linguistic variations (same answer could be written in different ways), subjective nature of assessment (multiple possible correct answers or no correct answer) and lack of consistency in human rating (non-binary scoring on an ordinal scale within a range). Earlier ASAG work requires supervisors to grade key concepts and their variations using concept mapping or to grade a fraction of student answers to train supervised learning algorithms.

Much work has been done in the field of ASAG in recent years. These works can be mainly divided into two categories. The first is supervised approach, in which the human graders first grade a few students' answers which in turn are used as training data for the algorithm. The second is unsupervised approach, in which different text similarity measures are used to compare the model answer and students' answers.

In this paper, we present an unsupervised approach, which consists of three models that score the students' responses individually. The first model starts with Parts-Of-Speech (POS) tagging of RA as well as SA, after which one word having a certain POS tag from RA is matched with all the words of SA having similar POS tags (for

example, a noun from RA is matched with all nouns from SA) and finally the maximum similarity score is taken. Such similarity score is obtained for every word of RA and finally an average of these scores is calculated to come up with the score that is to be awarded to the student.

The second model subjects the RA and SA individually to a parser which converts each of them into a set of dependency triples (a triple consists of three parts: the relation, the governor and the dependent words). The RA and SA triples are matched based on a set of pre-defined rules in which the relation part of the triple plays a significant role. Stanford POS tagger and dependency parser are used for the purpose of POS tagging and dependency triple extraction in this model.

The third and final model arranges the words of RA and SA in a two-dimensional matrix, with the words in RA (or SA; whichever has lesser number of words) in rows and words of SA (or RA) in columns. Once done, similarity score between a pair of words (one taken from a row and the other taken from a column) is measured. Once all the scores are obtained, the highest score is taken into consideration, after which the row and column to which this score belongs is struck off from being considered again. Then the word pair with the next highest similarity score from the rest of the matrix is considered, and the process is repeated until all the words in the rows are considered. Finally the average of these scores is taken over the number of rows to compute the final score that is to be awarded to the student.

For evaluation, we have calculated correlation between the obtained scores from the method and the scores awarded by human judges. Higher the correlation, the more efficient is the method to grade the students' answers close to the human judges.

The main motivation behind this research is to tap into the field of ASAG which is still full of possibilities. ASAG applications are quickly gaining popularity due to shifting of education from classrooms to online or e-learning centres, and this has provided us the motivation of coming up with another approach to solve the problem of ASAG. Our research is a small contribution to this vast field which is still growing and will continue to grow. Our method is not limited to a specific domain and can be used to evaluate responses from any courses, be it from humanities, science, commerce or any other course. Also no extra burden will be put on the grader, as all that needs to be done is to feed the model and students' responses into the method and the algorithm will take care of the rest of the evaluation process.

The rest of the paper is organized as follows. Section 2 discusses some related work reported in literature on this task. Section 3 presents the detailed description of our method. Section 4 illustrates the working of the proposed models with a running example. The experimental results and the details of the dataset have been provided in section 5. Section 6 provides the error analysis of the method. Finally section 7 draws the conclusion and provides some avenues for future work.

2 Related Work

Many approaches have already been reported in literature for the task of ASAG.

Gomma et al. [1] presented an unsupervised approach which uses text to text similarity measure. Different string-based and corpus-based similarity measures were tested separately and the results were then combined to obtain a final score.

Adams et al. [2] evaluated a variety of representative distributional semantics based approaches in the task of unsupervised grading and proposed an asymmetric method based on aligning word vectors that exploits properties of grading tasks. Aligning word vectors means moving the word vectors of one document in the vector space so that they can become word vectors of another document. This method allows words to move to multiple other words when a mismatch in document size occurs.

Alotaibi et al. [3] proposed a combination of different techniques to propose an approach for the task. In particular an integrated method was presented which uses Information Extraction (IE) and Machine Learning (ML) techniques. The purpose of the approach was to assign a score depending on the percentage of correctness in the student's response; not just classifying the resultant mark as correct or incorrect.

Roy et al. [4] proposed a technique which was based on the intuition that student answers to a question, as a collection, are expected to share more commonalities than any random collection of text snippets. If these commonalities can be identified from student answers then the same could be used to score them.

Sultan et al. [5] presented a fast and simple supervised method for the task. From the given RA and SA, they extracted a number of text similarity features and combined them with key grading-specific constructs.

Leacock et al. [6] presented the working of an automated short-answer marking engine called C-rater. If a set consisting of all the possible correct student responses is considered, then the C-rater scoring engine operates as a paraphrase recognizer that identifies members of this set, i.e. it identifies whether the student response is a paraphrase of the correct concept or not.

Basak et al. [7] presented a method that casts the ASAG task to an RTE task (Recognizing Textual Entailment), which determines whether the meaning of SA (or RA), is logically entailed by the RA (or SA): the better the entailment, the higher the score to be awarded to the student. To take the entailment decision between the RA and SA, they have used the rule set from one of their other works [8]. The matching rules are associated with different numerical scores and the total score to be awarded was calculated in a well-specified manner. We have incorporated the matching rules illustrated in [8] in our method for the purpose of grading the students' answers.

3 The Proposed Method

The proposed method consists of three different models, each model scoring the students' responses individually. The pre-processing steps for all the models are same.

3.1 Pre-processing

Each RA and SA is individually subjected to the pre-processing steps first, which include splitting words joined by a slash or hyphen (eg. "function/method" becomes "function method" and "run-time" becomes "run time") and removing unnecessary

symbols (the dataset contains certain HTML tags which need to be removed). We also removed the punctuation marks like ",", ";", "." and others as their absence did not impact the scores generated by the method, but their presence causes generation of unnecessary triples in Model 2. After that both RA and SA are converted to lowercases. The reason is that both Word2vec as well as fastText produce different similarity scores when the case is changed as compared to when the case is same.

Once pre-processing is done, the actual method of ASAG is invoked. The three models are explained in the subsequent subsections. One important thing to note is that for all the models, we have shown that the SA is evaluated with reference to the RA in cases where SA is concrete while RA is elaborate. However for some cases, it was found that the RA is concrete and to the point, while the SA is elaborate containing extra piece of information. For such cases, we reversed the roles of RA and SA, i.e., we scored the RA with reference to the SA. This is a part of the process in each model and not a separate model in itself, i.e., before the actual method is invoked; the length of the RA and SA is checked and the decision is made as to whether the SA is to be scored based on RA or the RA is to be scored based on SA. This is applicable for all the three models.

3.2 Model 1

The RA and SA are fed into Stanford POS-tagger [9]. As output from this phase the RA and SA with each word tagged with their corresponding POS are generated (eg. An input of "at the main function" will produce an output as "at/IN the/DT main/JJ function/NN", where NN signifies singular noun, JJ signifies adjective, IN signifies preposition or subordinating conjunction and DT signifies determiner).

Once the POS tagging is done, the response matching and scoring mechanism process gets started. Each word with a particular POS tag in RA is compared with every word in SA having an equivalent POS tag (eg. a word in RA with POS tag NN is matched with every word in SA with equivalent POS tags of NN, NNS, NNP or NNPS) and the maximum similarity score is considered. The similarity score between a given pair of words is determined by gensim's Word2vec model and Facebook's fastText model separately, where a 300 dimension vector of the words are generated in both models and the similarity score is obtained by measuring the cosine value of the angle between the two vectors.

Once the maximum score is obtained for each word in RA, the overall similarity score is calculated by taking the average of all the maximum scores over the number of words in the RA. This overall similarity score is then multiplied by the full marks of the question to obtain the marks that is to be awarded to the student.

3.3 Model 2

This model starts with parsing the RA and SA into a set of dependency triples by subjecting each of them individually into Stanford dependency parser [10].

Once the dependency parsing phase is over, the actual process of matching and scoring starts. The matching operation between the RA-SA triples is performed based on a set of predefined rules [8]. For simplicity, let us consider that the triples generated from RA and SA are of the forms $R_R(G_R, D_R)$ and $R_S(G_S, D_S)$ respectively,

where R_R and R_S signify the relation parts, G_R and G_S are the governor nodes, D_R and D_S are the dependent nodes. Here the ROOT node associated to a triple is ignored. The reason for excluding the root triple is that root is an extra relation generated just for the purpose of representing the word from which the parse tree is originated and has no such other significance. Its absence does not impact the score but its presence is found to lower the overall similarity score. The rules are described as follows:

3.3.1 Rule 1: If the SA triple $R_S(G_S, D_S)$ completely matches with an RA triple $R_R(G_R, D_R)$, i.e., R_S matches with R_R, G_S matches with G_R, and D_S matches with D_R, then a full score of 1 is assigned to the RA triple.

3.3.2 Rule 2: If the nodes D_S and G_S of an SA triple completely match with the nodes G_R and D_R respectively of an RA triple but the relations R_S and R_R do not match, then for the cases as listed in Table 1 (only a few equivalent relation pairs are shown), a full score of 1 is assigned to that RA triple.

3.3.3 Rule 3: If the nodes G_S and D_S of an SA triple match with the nodes G_R and D_R respectively of an RA triple but the relations R_S and R_R do not match, then for the few equivalent relation pairs of Table 2, a full score of 1 is assigned to the RA triple.

Table 1. Based on rules defined in [8]

R_R (Relation from RA triple)	**Equivalent R_S (Relation from SA triple)**
amod	dobj, nsubjpass, nsubj, nn
rcmod	nsubj, agent, dobj, nsubjpass
ccomp	advcl
vmod	nsubj

3.3.4 Rule 4: If the relation R_S exactly matches with the relation R_R and either the node pair G_S and G_R match exactly with each other or the node pair D_S and D_R match exactly, then the nodes D_S and D_R (or G_S and G_R) are compared with each other to obtain their similarity score. Then the average of these two scores (1 for exactly matched node pair and the other obtained on comparing the other node pair) is taken, which gives the score that is to be assigned to this RA triple.

Table 2. Based on rules defined in [8]

R_R (Relation from RA triple)	**Equivalent R_S (Relation from SA triple)**
nsubj	nsubjpass, agent, nn, csubj
amod	poss, nn, prep_in, appos, partmod
dobj	nsubjpass, iobj, prep_of
nn	prep_of, amod, nsubj

3.3.5 Rule 5: There are some relations that, although seem to be less important and insignificant, cannot be ruled out and need to be considered. These relations are mentioned in the following set:

{aux, auxpass, cop, det, expl, mark, nn, prt, predet, cc}

If these relations are ignored, it results in generating lower score. Therefore they should also be taken care of. One thing to note is that this rule is only applicable if both RA and SA contain at least one triple having relation belonging to the above set. If it happens that either only RA or only SA contains any triple with a relation from the above set, then this rule is not applied. The words G_S and G_R and the words D_S and D_R are compared to obtain their similarity score. The average of these two scores is taken, which gives the score that is to be assigned to the RA triple.

If an RA triple matches with multiple SA triples, then the maximum of all the maximum scores is considered and assigned to that triple. Once a score is assigned to each RA triple, the overall similarity score is calculated by taking the average of all the maximum scores over the number of triples in the RA. This overall similarity score is then multiplied by the full marks of the question to obtain the marks that is to be awarded to the SA.

3.4 Model 3

The RA and SA are broken down into its individual component words which are then arranged in the form of a two-dimensional matrix. Which of the RA or SA words will be arranged in the rows or columns depends on the length or the number of words in the answers. If the RA contains lesser number of words than the SA, its words will be arranged row-wise while those of SA will be arranged column-wise, and vice versa.

After the arrangement, each word from a row is matched with every word in the columns to obtain a similarity score. The gensim's Word2vec model and Facebook's fastText model used previously are also employed here to obtain the similarity score.

Next the scoring mechanism starts by taking the highest similarity score from the overall matrix, and then striking out the corresponding row and column. For example, a maximum score of say 0.98 is obtained at row 2 and column 3 of the matrix. Then this score is considered and it is made sure that none of the other scores belonging to row 2 and column 3 will be considered in the next iteration. This is done to make sure that one word from RA is matched to one and only one word of SA. The next maximum score from the rest of the matrix barring those contained in the stricken out row and column is obtained and the corresponding row and column are crossed off. The entire process is repeated until the maximum score from every row is considered.

Finally the average of these scores is computed over the number of rows to obtain an overall similarity score, which is then multiplied by the full marks carried by the question to obtain the marks that is to be awarded to the student. The idea of considering the maximum similarity score is based on n-queen's problem of greedy approach, where the target is to maximize the score that is to be awarded to the SA.

3.5 Special Case

Apart from the three models, we included a separate mechanism for scoring the special cases in which RA or SA consists of only a single word. For example, for certain questions, the RA is simply "push" whereas the SA contains the word "push" or one of its synonyms along with some extra information. For such cases, we compared this single word in RA, (or SA if the role of SA and RA are reversed) with

every word in SA (or RA), and took the maximum similarity score and used it to calculate the score that is to be awarded to the SA in consideration. However it is to be noted that this is not a separate model itself; rather it is part of all the three models and is applied only for the special case as mentioned here.

4 Running Example

Consider the following Q-RA-SA triple taken from Assignment 8 of the dataset:

Q: *What are the two main functions defined by a stack?*
RA: *push and pop*
SA: *pop and push*

The working is only shown with gensim's Word2vec. The approach is same when using fastText. Therefore its working is not shown here due to space constraints.

4.1 Model 1

Individual words along with their corresponding POS tags for both RA and SA are shown in Table 3. The scores obtained when the model is run with Word2vec are shown in Table 4.

Now “and” from SA only matches with one word “and” in RA with similarity score of 1.0, so this score is considered. For “push” from SA, there are two matching words "push" and "pop" in RA with similarity scores of 1.0 and 0.185 respectively among which 1.0 is maximum, therefore this score is considered. For “pop” from SA also, there are two matching words "pop" and "push" in RA with respective similarity scores of 1.0 and 0.185 among which 1.0 is maximum and this score is considered. Now all these similarity scores are obtained, the average score is calculated as:

$$\text{Score (Word2vec)} = (1.0 + 1.0 + 1.0) / 3 = 1.0 . \quad (1)$$

Table 3. Words with POS tags

Words in RA	POS tags	Words in SA	POS tags
push	NN	pop	NN
and	CC	and	CC
pop	NN	push	NN

Table 4. Scores obtained with Word2vec

	pop (NN)	and (CC)	push (NN)
push(NN)	0.185	-	1.0
and(CC)	-	1.0	-
pop(NN)	1.0	-	0.185

Since the question carries a full mark of 5, the grade to be awarded to this SA is calculated as:

$$\text{Grade (Word2vec)} = 1.0 \times 5 = 5.0 . \quad (2)$$

Both the human graders have awarded score of 5 to this response, and the obtained score of 5.0, is same as the scores awarded by the human judges.

4.2 Model 2

The RA and SA dependency triples of the above Q-RA-SA triple are provided in Table 5, while the matched token pairs, the rule based on which they are matched, their similarity scores and the average similarity scores are provided in Table 6. Since the triple pairs SA1-RA1 and SA4-RA4 are associated to ROOT node and labeled by root relation, they are not taken into consideration for assignment of scores.

Table 5. RA and SA dependency triples

RA triples	SA triples
RA1: root(ROOT-0, push-1)	SA1: root(ROOT-0, pop-1)
RA2: cc(push-1, and-2)	SA2: cc(pop-1, and-2)
RA3: conj:and(push-1, pop-3)	SA3: conj:and(pop-1, push-3)
RA4: root(ROOT-0, pop-3)	SA4: root(ROOT-0, push-3)

Table 6. Assignment of scores

SA triple matched with RA triple	Token pair being matched	MR	SS	AS
SA2- RA2	push→pop	4	0.185	0.593
	and→and		1.0	
SA3-RA3	push→push	2	1.0	1.0
	pop→pop		1.0	

Here, MR signifies the matching rule number, SS signifies the similarity score of the tokens being matched and AS signifies the average of the similarity scores between the corresponding governors and dependents of RA and SA, which is the score that to be assigned to the triple.

Here every triple of RA is getting matched to only one triple of SA, therefore obtaining maximum score for each triple of RA is not required since there is only one.

The average similarity score is calculated as:

$$\text{Score (Word2vec)} = (0.593 + 1.0) / 2 = 0.797 . \quad (\mathbf{5})$$

The grade to be awarded to the SA is:

$$\text{Grade (Word2vec)} = 0.797 \text{ x } 5 = 3.985 = 4.0 . \quad (\mathbf{6})$$

Both the human graders have awarded score of 5 to this response, and the obtained score of 4.0, is significantly closer to the scores awarded by the human judges.

4.3 Model 3

At first the RA-SA words are arranged in the form of a two-dimensional matrix. If the RA contains lesser number of words than SA, then RA words are arranged in rows while the SA words are arranged in columns. On the other hand, if the SA contains lesser number of words than RA, then SA words are arranged in rows while the RA

words are arranged in columns. Once this arrangement is done, the semantic similarity scores between each RA-SA word pair is calculated and stored in the matrix.
The scores obtained when the model is run with Word2vec are arranged in the form of a two-dimensional matrix as shown in Table 7. The approach is same when using fastText with the difference in the generated similarity scores and the overall score. Therefore its working is not shown here.

The algorithm to find the highest similarity score starts at row 0 and column 2. Since it contains the overall highest score with value 1.0, it is taken and the row 0 and column 2 are struck off. The updated matrix is shown in Table 8.

Table 7. Scores obtained with Word2vec

	pop	**and**	**push**
push	0.185	0.034	1.0
and	0.047	1.0	0.034
pop	1.0	0.047	0.185

Table 8. Updated after first iteration

	pop	**and**	**push**
push	-	-	-
and	0.047	1.0	-
pop	1.0	0.047	-

In the next iteration the next highest score is taken, which is again 1.0 at row 1 and column 1, so it is taken and the corresponding row and column are struck off.

In the next and final iteration the next highest score from the remaining matrix is taken, which is again 1.0 at row 2 and column 0, so it is taken and the corresponding row and column are struck off.

Since all the rows have been considered the algorithm stops and the average similarity score is calculated as:

$$\text{Score (Word2vec)} = (1.0 + 1.0 + 1.0) / 3 = 1.0 . \quad (7)$$

The grade to be awarded to the SA is:

$$\text{Grade (Word2vec)} = 1.0 \times 5 = 5.0 . \quad (8)$$

Both the human graders have awarded scores of 5 to this response, and the obtained score of 5.0, is exactly the same as the scores awarded by the human judges.

5 Experiments and Results

Experiments were carried out for all three of our models on a standard computer science dataset consisting of questions, reference answers and student responses taken from an undergraduate computer science course [10][1]. Table 9 presents a summary of the dataset used here for the experimental purpose. The dataset consists of a total of 12 assignments containing different number (9 assignments have 7 questions, 1 has 4 and 2 have 10) of questions. Each question (Q) is provided with a reference answer (RA) as well as a set of student answers (SA) as well. The breakdown of the number of questions as well as the number of student responses for each question and for each

[1] **web.eecs.umich.edu/~mihalcea/downloads/**

assignment is provided in the Table 9. Each student response is evaluated by two human graders and the average of their scores is also provided.

The performances of all the three models were tested on the 12 assignments and the scores produced by the method is compared with the scores of both the examiners as well as their average to come up with the correlation which are provided in the Table 10 and Table 11. Table 10 presents the scores obtained using gensim's Word2vec model trained on Google-News corpus, while Table 11 presents the scores obtained using Facebook's fastText trained on Wikipedia articles. Each of the tables shows the scores using all three of our models. *Ex1* and *Ex2* indicate the correlation between the scores produced by our method and the scores awarded by Examiner 1 and Examiner 2 respectively. *Avg* indicates the correlation between the scores produced by the method and the average scores of the two examiners.

As observed from the tables, Model 1, 2 and 3 exhibit the best performances, with correlation of 0.608, 0.434 and 0.646, for Assignments 1, 9 and 4 respectively using Word2vec on Google-News corpus. On the other hand, the three models exhibit the best performances, with correlation of 0.625 0.601 and 0.666 for Assignments 1, 6 and 1 respectively using fastText on English Wikipedia corpus. The overall correlation over the entire assignment was also computed and it reaches the highest value of 0.805 for Model 3 with Word2vec on Google-News corpus.

Table 9. Statistics of the dataset being used

Assignment No.	No. Of Questions (Q)	No. of SA per Q	Total no. of SA
1	7	29	203
2	7	30	210
3	7	31	217
4	7	30	210
5	4	28	112
6	7	26	182
7	7	26	182
8	7	27	189
9	7	27	189
10	7	24	168
11	10	30	300
12	10	28	280
Total	87	-	2442

Table 10. Scores with Word2vec on Google-News Corpus

Assign-ment	Model 1			Model 2			Model 3		
	Ex 1	**Ex 2**	**Avg**	**Ex 1**	**Ex 2**	**Avg**	**Ex 1**	**Ex 2**	**Avg**
1	0.557	0.565	**0.608**	0.325	0.373	0.377	0.555	0.554	0.602
2	0.218	0.393	0.319	0.041	0.188	0.113	0.224	0.396	0.324
3	0.416	0.426	0.475	0.045	0.045	0.051	0.297	0.211	0.299
4	0.468	0.576	0.546	0.239	0.149	0.223	0.534	**0.646**	0.619
5	0.548	0.357	0.539	0.032	0.017	0.03	0.532	0.415	0.552

6	0.426	0.426	0.579	0.241	0.405	0.365	0.323	0.475	0.453
7	0.375	0.497	0.468	0.072	-0.075	0.009	0.353	0.437	0.427
8	0.419	0.425	0.463	0.312	0.376	0.365	0.32	0.259	0.331
9	0.421	0.334	0.427	0.411	0.38	**0.434**	0.521	0.424	0.531
10	0.423	0.383	0.438	0.139	-0.008	0.103	0.294	0.252	0.3
11	0.374	0.434	0.425	0.219	0.265	0.252	0.358	0.486	0.429
12	0.386	0.438	0.425	0.156	0.187	0.169	0.239	0.269	0.251
Overall	0.652	0.785	0.448	0.395	0.466	0.221	0.639	**0.805**	0.4

Table 11. Scores with fastText on Wikipedia Corpus

Assign-ment	**Model 1**			**Model 2**			**Model 3**		
	Ex 1	**Ex 2**	**Avg**	**Ex 1**	**Ex 2**	**Avg**	**Ex 1**	**Ex 2**	**Avg**
1	**0.625**	0.582	0.656	0.483	0.477	0.521	0.632	0.594	**0.666**
2	0.253	0.371	0.33	0.222	0.397	0.324	0.351	0.478	0.442
3	0.434	0.416	0.484	0.395	0.315	0.413	0.429	0.347	0.451
4	0.524	0.472	0.545	0.319	0.269	0.325	0.535	0.496	0.561
5	0.398	0.346	0.427	0.109	0.068	0.106	0.471	0.444	0.519
6	0.356	0.593	0.536	0.325	**0.601**	0.52	0.368	0.624	0.559
7	0.421	0.539	0.518	0.309	0.357	0.361	0.386	0.503	0.478
8	0.443	0.429	0.482	0.329	0.281	0.346	0.364	0.342	0.393
9	0.244	0.167	0.239	0.154	0.112	0.153	0.223	0.162	0.221
10	0.46	0.402	0.472	0.408	0.339	0.413	0.491	0.429	0.503
11	0.559	0.561	0.612	0.479	0.531	0.537	0.558	0.577	0.615
12	0.456	0.459	0.488	0.294	0.238	0.282	0.398	0.377	0.414
Overall	0.686	0.783	0.461	0.609	0.713	0.335	0.687	**0.801**	0.456

Fig. 1 and Fig. 2 represent the comparison of correlation for the three models using Word2vec and fastText similarity measures respectively. For graphical representation, only the *Avg* of the scores awarded by the two Examiners *Ex1* and *Ex2* is considered. From Fig. 1 and Fig. 2, it is observed that except for Assignment 9, all three models perform better when using fastText instead of Word2vec. The reason is that fastText can even build vectors for words that are not in the corpus being used by using the vectors for the components that make up the unknown word. It does so by breaking up the word into n-gram characters (the value of n is decided by the fastText algorithm and not by us), and then using the vector representations of these n-gram components to build the vector for the word. Hence presence of words such as *Enqueue*, which has important significance in computer science terminology but not in plain English, does not negatively affect the score to be awarded to the student.

One more thing to note is that the correlation values obtained for Assignments 5 and 7 are lower for Model 2 when Word2vec is used as compared to fastText. The reason behind this is the different working principles of Word2vec and fastText. Word2vec uses the whole word to generate its corresponding vector, while fastText breaks down the word into a group of n-grams, then generates vectors for these

individual n-grams and then finally adds them up to come up with a vector for the original word. Therefore when a word that is not present in the corpus is supplied to Word2vec model, it generates an erroneous resultant vector which results in generation of a lower score when the word is compared with any other word.

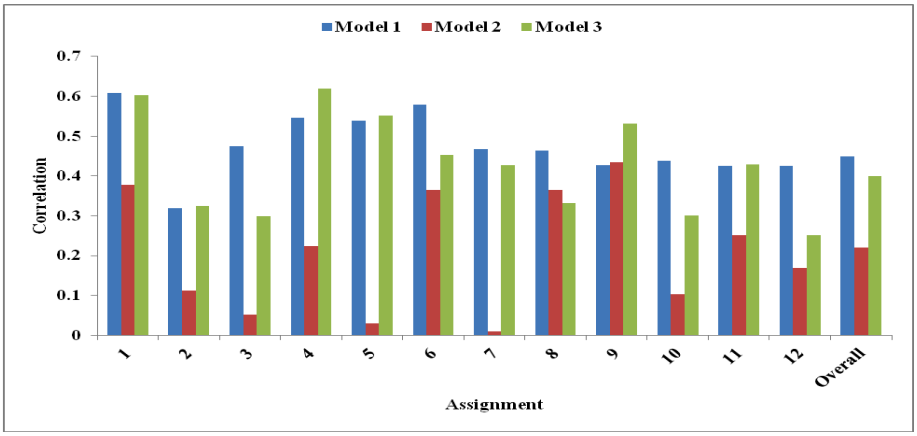


Fig. 1 Comparison of Correlation using Word2vec

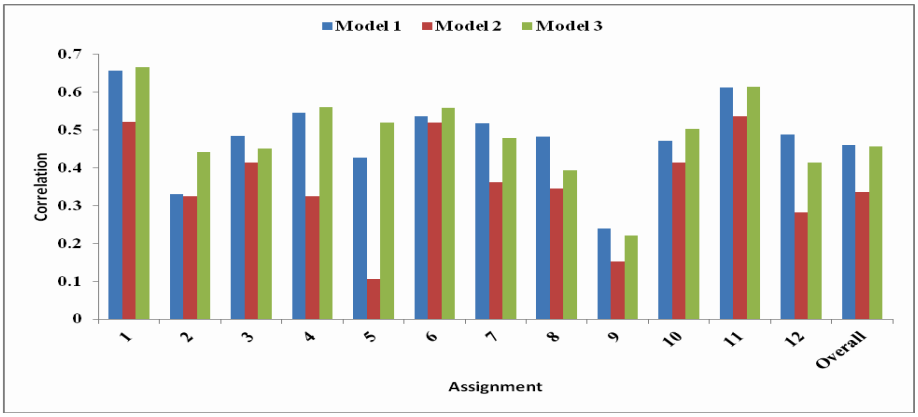


Fig. 2 Comparison of Correlation using fastText

However this is not the case with fastText, as even if the input word is not present in its corpus, it still breaks the word down into a group of n-grams where the n-grams might be present for some other words in the corpus and thus can be used to generate a vector for this word. For example, the pair of words (n, array) when checked with Word2vec generated a similarity score of 0.010 while for the same fastText generated a score of 0.231. Frequent occurrence of such word pairs lower the overall similarity score, which in turn lower the grade to be awarded to the SA. Such cases are found in much more quantity in Assignments 5 and 7 compared to the rest of the dataset.

6 Error Analysis

Even though our models produce scores quite close to those of the human scorers, in some cases they produce low scores as well. One such case is given below, which is taken from Assignment 8 of the dataset:

Q: *What operations would you need to perform to find a given element on a stack?*
RA: *Pop all the elements and store them on another stack until the element is found, then push back all the elements on the original stack.*
SA: *StackPush() StackPop() StackIsEmpty()*

Here the student has answered the question by giving the name of the operations i.e. instead of writing "pop an element from the stack" the student has written this operation in terms of function name "*StackPop()"*. Now a human scorer easily understands that the function signifies a pop operation, however the method requires a little bit of context to understand this concept. The same goes for the other two functions as well, i.e. "*StackPush()"* signifies pushing an element into the stack and "*StackIsEmpty()"*signifies checking whether the stack is empty or not. To handle such cases, the context needs to be fed separately into the method in order to explain the concept in some way.

7 Conclusions and Future Work

As evident from the correlation between the scores obtained from the proposed method and the scores awarded by the examiners, all three of our models have proven to be quite efficient in assigning scores to the student answers sufficiently close to that awarded by the human examiners. Given a reference answer (RA), our models assign a similarity score (in the range [0, 1]) to the student answer (SA), which is then scaled up by multiplying the score with the full marks of the question (Q).

Experiments were carried out on all three of our models on a standard ASAG dataset and the correlation obtained between our scores and those awarded by the human examiners reveals that our models can competently award the students' responses with scores sufficiently close to those of the human examiners. It is important to note that the performance of our method in terms of correlation surpasses the highest state-of-the-art result reported on this dataset so far.

In some cases, we observed that the RA has conveyed the answer in very few words, i.e. in a gist, while the student has conveyed almost the same answer with more number of words by providing some extra information, which may or may not be related to the actual answer. In such cases, if the SA is evaluated with respect to the corresponding RA, this extra piece of information contained within SA may not get matched to that provided in the RA, resulting in assignment of a lower score to the SA than it should have been actually awarded. Thus, for such cases reversing the role of RA and SA, i.e. evaluating the RA with respect to the SA, is found to be justified as the models generate fairly higher scores nearer to the ones awarded by the human examiners. However, for such cases it is assumed that the extra piece of information

provided by the student is correct, and therefore the provision of penalising the student response for extra, incorrect or irrelevant information is currently out of the scope of the proposed work.

In future, we will augment the method with some advance mechanism to find out whether the extra piece of information is incorrect or not in relevance to the context of the answer, which will help us with the decision of penalising the SA or not.

Apart from the rules that have been used in our method for the purpose of matching the RA and SA triples, the efficiency of our method is upper bounded by the similarity scores produced by the corpus-based semantic similarity measures Word2vec and fastText. The similarity scores produced from these modules play a pivotal role in generating scores nearer to the human graders. Naturally incorrect scores produced by them adversely impact the performance of the models.

References

1. Gomaa, W.H., Fahmy, A. A.: Short Answer Grading Using String Similarity and Corpus-Based Similarity, International Journal of Advanced Computer Science and Applications, Vol. 3, no. 11, (2012).
2. Adams, O., Roy, S., Krishnapuram, R.: Distributed Vector Representations for Automatic Short Answer Grading, Proceedings of the 3rd Workshop on Natural Language Processing Techniques for Educational Applications, pp. 20–29, (2016).
3. Alotaibi, S. T., Mirza, A.A.: HYBRID Approach For Automatic Short Answer Marking, in proceedings of 2012 Southwest Decision Sciences Institute Conference (SWDSI), pp. 581-589, (2012).
4. Roy, S., Dandapat, S., Nagesh, A., Narahari, Y.: Wisdom of students: A consistent automatic short answer grading technique, In Proceedings of 13th ICON-2016, pp. 178, (2016).
5. Sultan, M.A., Salazar, C., Sumner, T.: Fast and easy short answer grading with high accuracy, In: NAACL: HLT, San Diego, California, pp. 1070–1075, (2016).
6. Leacock, C., Chodorow, M.: C-rater: Automated Scoring of Short-Answer Questions, *Computers and the Humanities*, 37(4), pp. 389-405, (2003).
7. Basak, R., Naskar, S.K., Gelbukh, A.: Short-answer grading using textual entailment. Journal of Intelligent & Fuzzy Systems, Vol. 36, no. 5, pp. 4909-4919, (2019).
8. Basak, R., Naskar, S.K., Pakray, P., Gelbukh, A.:Recognizing Textual Entailment by Soft Dependency Tree Matching, Computacion y Sistemas, Vol. 19, no. 4, pp. 685–700, (2015).
9. Manning, C.D., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S. J., McClosky, D.: The Stanford CoreNLP Natural Language Processing Toolkit, In Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 55-60, (2014).
10. Mohler, M., Bunescu, R., Mihalcea, R.: Learning to grade short answer questions using semantic similarity measures and dependency graph alignments, HLT: 49th Annual Meeting of the ACL: Human Language Technologies, Vol, 1, pp. 752-762, (2011).